

GUÍA COMPLETA

Roll 3

Restaurantes

Actividad Integradora · Sprint 1

Misión del rol Implementar y demostrar la HU 2 Crear Restaurante, aportar la integración de autorización de la HU 5 y producir el modelo relacional y la documentación técnica correspondiente.

Responsable	Coordinación	Alcance actual
Integrante 3	Molly Lecompte · Scrum Master	Exclusivamente Sprint 1: HU 2 y apoyo en HU 5

1. Alcance exacto de esta guía

Esta versión se limita al Entregable 1. El Compañero B es responsable completo de la creación de restaurantes y colabora con el Compañero A para que el endpoint quede protegido por autenticación y rol. Las decisiones marcadas como propuesta técnica deben ajustarse a la arquitectura acordada por el equipo.

Asignación	Responsabilidad	Resultado
HU 2	Responsable completo	El administrador crea un restaurante asociado a un propietario válido.
HU 5	Colaboración técnica	El endpoint de restaurante solo admite al rol ADMINISTRADOR autenticado.
Documentación	Responsable principal	Modelo relacional inicial y documentación de Restaurante.

Fuera de alcance Habilitar platos, listar restaurantes o menús, pedidos, trazabilidad, notificaciones y métricas pertenecen a entregas posteriores.

2. Productos que debe entregar:

- ☐ Código funcional de HU 2.
- ☐ Entidad y persistencia de Restaurante.
- ☐ Validaciones de nombre, NIT, teléfono y propietario.
- ☐ Integración con Usuarios para comprobar el rol del propietario.
- ☐ Protección del endpoint para ADMINISTRADOR.
- ☐ Pruebas unitarias y de integración.
- ☐ Documentación del endpoint Crear Restaurante.
- ☐ Modelo relacional inicial de usuarios, restaurantes, platos y autenticación, coordinado con el equipo.
- ☐ Aporte al diagrama de clases de Plazoleta.
- ☐ Colección de API y evidencias para sustentación.
- ☐ Rama, commits y Pull Request verificables.

3. Paso 0 · Alineación con el equipo

1. Revisar con Molly los criterios exactos de HU 2 y la definición de terminado.
2. Acordar con Compañero A el contrato para consultar si un usuario existe y tiene rol PROPIETARIO.
3. Acordar con Compañero A cómo llegará la identidad y el rol ADMINISTRADOR al endpoint.
4. Acordar con Compañero C la relación Restaurante-Plato para el modelo relacional y el diagrama de clases.
5. Definir nombres de campos, tipos de datos, identificadores y formato estándar de errores.
6. Dividir HU 2 en análisis, dominio, persistencia, integración, API, seguridad, pruebas y documentación.
7. Registrar tareas, estimación, dependencias y riesgos en Kanban.

Evidencia esperada Tarjeta HU 2 desglosada, estimada, asignada y con cada criterio de aceptación visible.

4. Paso 1 · Preparar repositorio y entorno

8. Clonar el repositorio.
9. Actualizar develop.
10. Verificar que el proyecto compile y las pruebas base funcionen.
11. Crear o revisar el microservicio de Plazoleta.
12. Configurar persistencia y variables sin subir secretos.
13. Crear la rama HU 2 desde develop.

```
git checkout develop
git pull origin develop
git checkout -b feature/HU-02-crear-restaurante
```

Estructura propuesta

```
plazoleta/
  domain/      # Restaurante, reglas y excepciones
  application/ # caso de uso, DTO y puertos
  infrastructure/ # API, persistencia y cliente de Usuarios
  config/      # configuración transversal
  test/        # pruebas unitarias e integración
```

Checklist de entorno

- ☐ Aplicación inicia.
- ☐ Base de datos responde.
- ☐ No hay secretos versionados.
- ☐ Rama creada correctamente.
- ☐ Tarjeta HU 2 en En progreso.

5. Paso 2 · Analizar HU 2

Objetivo obligatorio Un administrador crea un restaurante con datos válidos y asociación a un usuario que realmente tenga rol propietario.

5.1 Campos y reglas obligatorias

Campo	Regla
Nombre	Obligatorio; puede contener números, pero no puede estar compuesto solo por números.
NIT	Obligatorio y únicamente numérico.
Dirección	Obligatoria.
Teléfono	Obligatorio, máximo 13 caracteres y puede incluir el símbolo +.
URL del logo	Obligatoria.

Campo	Regla
Id del propietario	Obligatorio; debe corresponder a un usuario con rol PROPIETARIO.

5.2 Seguridad aplicable

- Solo un ADMINISTRADOR autenticado puede crear restaurantes.
- Un usuario autenticado con otro rol debe recibir acceso denegado.
- El id recibido no debe aceptarse sin comprobar el rol en Usuarios.

5.3 Diseño previo

14. Definir método, ruta, DTO de entrada, DTO de salida y errores.
15. Modelar Restaurante y la referencia al propietario.
16. Definir el puerto de persistencia.
17. Definir el puerto de consulta al microservicio de Usuarios.
18. Definir excepciones: datos inválidos, propietario inexistente, rol incorrecto y acceso denegado.
19. Acordar el formato de respuesta con el resto del equipo.

6. Paso 3 · Modelar Restaurante

6.1 Entidad de dominio

20. Crear la entidad Restaurante sin dependencia directa del controlador.
21. Incluir nombre, NIT, dirección, teléfono, URL del logo e id del propietario.
22. Mantener las validaciones de negocio en componentes reutilizables o en el caso de uso.
23. Evitar que la entidad dependa de detalles de HTTP o de la base de datos.

6.2 Persistencia

24. Crear el modelo o entidad persistente.
25. Definir la tabla y tipos de datos acordados.
26. Crear el repositorio para guardar y consultar restaurantes.
27. Implementar mapeos entre dominio y persistencia.
28. Preparar migración o script de base de datos si el equipo lo utiliza.

6.3 Relación con propietario

En una arquitectura de microservicios, la forma exacta de representar la relación debe acordarse. Como propuesta técnica, Plazoleta puede conservar el identificador del propietario y validar su existencia y rol mediante el contrato de Usuarios, sin duplicar toda la entidad Usuario.

7. Paso 4 · Implementar validaciones

29. Rechazar solicitudes con campos obligatorios ausentes.
30. Validar que NIT contenga únicamente dígitos.
31. Validar el teléfono según el máximo de 13 caracteres y el símbolo + permitido.
32. Validar que el nombre no sea únicamente numérico.

33. Validar los demás formatos conforme al contrato acordado, sin inventar restricciones no incluidas en los anexos.
34. Devolver mensajes de error consistentes y útiles.

Matriz de validación

Caso	Entrada	Resultado
NIT válido	Solo dígitos	Continúa.
NIT inválido	Incluye letras o símbolos	Rechazo.
Teléfono válido	Formato permitido, máximo 13	Continúa.
Teléfono largo	Más de 13 caracteres	Rechazo.
Nombre válido	Letras o combinación alfanumérica	Continúa.
Nombre inválido	Solo números	Rechazo.
Campo ausente	Falta un obligatorio	Rechazo.

8. Paso 5 · Integrar con Usuarios

35. Solicitar al Compañero A el contrato acordado para consultar usuario y rol.
36. Implementar un puerto de aplicación para validar propietario.
37. Crear el adaptador o cliente correspondiente.
38. Enviar el id del propietario recibido.
39. Comprobar que el usuario existe.
40. Comprobar que el rol sea PROPIETARIO.
41. Rechazar la creación cuando no exista o tenga otro rol.
42. Controlar fallos de comunicación de forma coherente y reportarlos como bloqueo si afectan la entrega.
43. Probar el contrato de integración con datos controlados.

Regla crítica La asociación no es válida solo porque el id exista; el anexo exige verificar que el usuario tenga rol propietario.

9. Paso 6 · Implementar el caso de uso

44. Recibir el comando o DTO validado.
45. Solicitar la validación del propietario.
46. Construir la entidad Restaurante.
47. Persistir el restaurante.
48. Devolver una respuesta sin información innecesaria.
49. Propagar errores de negocio mediante el esquema estándar.

Flujo lógico

```
Solicitud ADMINISTRADOR
-> validar campos
-> validar propietario en Usuarios
-> construir Restaurante
-> guardar en Plazoleta
-> devolver respuesta
```

10. Paso 7 · Crear y proteger el endpoint

50. Crear el controlador de creación de restaurante.
51. Delegar la lógica al caso de uso.
52. Aplicar validación del DTO.
53. Recibir o resolver la identidad autenticada según el mecanismo acordado.
54. Coordinar con Compañero A la regla que permite únicamente ADMINISTRADOR.
55. Comprobar acceso denegado para PROPIETARIO, EMPLEADO y CLIENTE.
56. Asegurar respuestas diferenciadas para datos inválidos, no autenticado y no autorizado.

Matriz de autorización

Actor	Crear restaurante
Administrador autenticado	Permitido
Propietario	Denegado
Empleado	Denegado
Cliente	Denegado
Sin autenticar	Denegado

11. Paso 8 · Probar HU 2

Prueba	Condición	Resultado esperado
Creación válida	Administrador, datos y propietario válidos	Restaurante creado.
Propietario inexistente	Id no encontrado	Solicitud rechazada.
Rol asociado incorrecto	Id existe, pero no es PROPIETARIO	Solicitud rechazada.
NIT inválido	Contiene caracteres no permitidos	Solicitud rechazada.
Teléfono inválido	Supera 13 caracteres	Solicitud rechazada.
Nombre inválido	Compuesto solo por números	Solicitud rechazada.
Campo ausente	Falta un obligatorio	Solicitud rechazada.

Prueba	Condición	Resultado esperado
Actor incorrecto	No es ADMINISTRADOR	Acceso denegado.
Sin credencial	No autenticado	Acceso denegado.

Tipos de prueba que debe conservar

- ☐ Unitarias de validadores.
- ☐ Unitaria del caso de uso con dobles de los puertos.
- ☐ Integración de persistencia.
- ☐ Integración del controlador.
- ☐ Integración con el contrato de Usuarios.
- ☐ Autorización con y sin rol ADMINISTRADOR.
- ☐ Colección de API con casos válido e inválidos.

12. Paso 9 • Modelo relacional

El Sprint 1 exige un modelo relacional inicial para usuarios, restaurantes, platos y autenticación. El Compañero B coordina este artefacto, pero debe incorporar los datos confirmados por los responsables de Usuarios y Platos.

57. Solicitar al Compañero A la estructura final de Usuario, Rol y elementos de autenticación persistidos.
58. Solicitar al Compañero C la estructura final de Plato y Categoría.
59. Incluir Restaurante con todos los campos de HU 2.
60. Representar claramente claves primarias y relaciones.
61. Representar la asociación entre propietario y restaurante conforme a la decisión de microservicios.
62. Representar la asociación entre restaurante y platos.
63. Revisar cardinalidades con todo el equipo.
64. Asegurar coherencia entre diagrama, migraciones y código.
65. Versionar el archivo fuente y una versión visible para la entrega.

Importante El responsable coordina el modelo completo, pero cada integrante debe validar las entidades de su propio módulo.

13. Paso 10 • Documentación técnica

- Descripción de la responsabilidad de Restaurante dentro de Plazoleta.
- Diagrama de clases del componente Restaurante.
- Campos, tipos y reglas de la entidad.
- Contrato del endpoint Crear Restaurante.
- Ejemplos de solicitud, respuesta y errores.
- Contrato de validación con Usuarios.
- Matriz de autorización.
- Modelo relacional consolidado.
- Evidencias de pruebas.
- Decisiones técnicas y supuestos identificados como tales.

14. Paso 11 · Git y Pull Request

Rama

```
feature/HU-02-crear-restaurante
```

Ejemplos de commits

```
feat(plazoleta): modelar restaurante [HU-02]
feat(plazoleta): validar datos de restaurante [HU-02]
feat(plazoleta): validar rol propietario [HU-02]
test(plazoleta): probar creación de restaurante [HU-02]
docs(plazoleta): actualizar modelo relacional [HU-02]
```

Cada integrante debe realizar al menos 3 commits por semana en días distintos. Deben ser avances reales y verificables.

Checklist del Pull Request

- ☐ Rama creada desde develop actualizada.
- ☐ HU 2 identificada en título y descripción.
- ☐ Todos los criterios cubiertos.
- ☐ Pruebas ejecutadas y documentadas.
- ☐ Integración con Usuarios comprobada.
- ☐ Autorización ADMINISTRADOR comprobada.
- ☐ Modelo y documentación actualizados.
- ☐ Sin secretos ni código de depuración.
- ☐ Revisión solicitada y observaciones resueltas.

15. Participación Scrum

Elemento	Aporte del Compañero B
Sprint Planning	Estimar HU 2 y desglosar tareas y dependencias.
Daily Scrum	Reportar avance, próximo paso, pruebas y bloqueos.
Kanban	Actualizar el estado real y criterios.
Sprint Review	Demostrar creación y validaciones de restaurante.
Sprint Retrospective	Identificar deuda técnica y acción correctiva.
Sustentación	Explicar modelo, integración, pruebas y seguridad.

16. Guion de demostración en vivo

66. Iniciar Plazoleta y verificar disponibilidad.
67. Autenticarse como administrador con apoyo del Compañero A.

- 68. Crear un restaurante con propietario válido.
- 69. Mostrar el registro persistido y su asociación.
- 70. Demostrar rechazo con id inexistente.
- 71. Demostrar rechazo cuando el usuario no es propietario.
- 72. Demostrar rechazo de NIT inválido.
- 73. Demostrar rechazo de teléfono con más de 13 caracteres.
- 74. Demostrar rechazo de nombre solo numérico.
- 75. Demostrar acceso denegado con otro rol y sin autenticación.
- 76. Mostrar pruebas, commits, Pull Request y modelo relacional.

17. Lista maestra de terminado

Funcionalidad

- ☐ Campos obligatorios implementados.
- ☐ NIT validado.
- ☐ Teléfono validado.
- ☐ Nombre no admite solo números.
- ☐ Propietario existente y con rol correcto.
- ☐ Restaurante persistido y asociado.

Seguridad y pruebas

- ☐ Solo ADMINISTRADOR crea restaurantes.
- ☐ Casos positivos y negativos superados.
- ☐ Integración con Usuarios probada.
- ☐ Errores consistentes.

Evidencias

- ☐ Mínimo 3 commits semanales en días distintos.
- ☐ Rama feature y Pull Request revisado.
- ☐ Kanban actualizado.
- ☐ Modelo relacional actualizado.
- ☐ Diagrama y contrato documentados.
- ☐ Colección de API y sustentación preparadas.

18. Riesgos y medidas preventivas

Riesgo	Medida
Aceptar cualquier id como propietario	Consultar Usuarios y comprobar rol.
Fallo de contrato entre servicios	Acordar contrato temprano y versionarlo.
Autorizar solo desde el controlador	Mantener controles coherentes con la configuración de seguridad.
Modelo relacional desactualizado	Actualizarlo en el mismo Pull Request.
Duplicar entidad Usuario en Plazoleta	Guardar solo la referencia acordada y documentar la decisión.

Riesgo	Medida
Desarrollar HU futuras	Mantener el alcance en HU 1 a HU 5.